VEXON DEVELOPER HANDBOOK · GUIDE 01

VEXON Project Overview

The whole system on one map: apps, services, data, security and how the pieces talk to each other

Written for	A B.Tech student who built VEXON with AI help and now wants to understand and maintain it alone
Project	VEXON / CampusConnect (repository: this folder's parent project)
Evidence	Generated from a direct inspection of the repository source code
Repository state	branch fix/csrf-legacy-cookie, commit af623fb, plus 126 uncommitted working-tree change(s)
Generated	16 October 2026, 21:11
Rebuild	python docs/developer-handbook/build_handbook.py

Accuracy rule: every statement about VEXON in this guide is based on files in the repository. Anything that could not be confirmed is labelled 'unverified', 'planned' or 'hypothetical'. No secrets, passwords or real student data are reproduced.

Contents

How to use this guide . . . 3
What VEXON is . . . 3
Feature areas found in the code . . . 3
The repository at a glance . . . 4
Architecture . . . 5
The big picture . . . 5
How a request moves through the system . . . 6
How the frontend talks to the backend . . . 6
How the backend talks to the database . . . 7
Authentication and authorization . . . 7
Signing in (authentication) . . . 7
Permissions (authorization) . . . 8
Real-time communication (Socket.IO) . . . 8
Mobile and iOS architecture . . . 9
How the apps share the backend . . . 9
The three mobile code bases . . . 9
External services . . . 10
Important dependencies . . . 10
Running VEXON locally . . . 11
What the server does when it starts . . . 11
Known documentation and code mismatches . . . 11
Glossary . . . 12
Check your understanding . . . 12
Source references . . . 13

How to use this guide

This is the first technical guide in the handbook. It describes how VEXON is built *today*, based on what is in the repository. It does not describe what the project was meant to become. Every claim points to a real file. When a file path appears in `this style`, you can open it in VS Code and check the claim yourself. Do that often. It is the most important habit this handbook teaches.

Evidence status

Five labels are used across the handbook:

- **Implemented and verified**: the code exists and an automated test that was actually run during this audit (16 October 2026) exercises it.
- **Implemented, not fully tested**: the code exists, but no test that ran covers the whole behaviour.
- **Partially implemented**: some of the feature exists; important parts are missing or switched off.
- **Planned, not implemented**: mentioned in documents or prompts, but no working code exists.
- **Unable to verify**: could not be checked from this Windows machine (for example the iOS app, which needs a Mac).

What VEXON is

VEXON is a **college campus management platform**. One backend serves three kinds of clients: a web dashboard, Android apps and an early iOS app. Students, faculty, HODs, the principal, security staff, wardens, transport staff and alumni all sign in to the same system. Each role sees only what it is allowed to see.

You will meet three names in the code. They all refer to the same project:

Name	Where you see it	Meaning
VEXON	`README.md` ("by VEXON TEAM"), `mobile-ios/README.md`, Kotlin package `com.vexon.campus`	The team and product name
CampusConnect	root `package.json` (`campusconnect`), `server/package.json`, server log lines	The original project and package name
JNN	Kotlin app label "JNN (Kotlin)", `render.yaml` service `jnn-api`, the AI "JNN Study Assistant"	The college brand shown to users

Feature areas found in the code

The backend registers about **300 REST routes** in `server/src/routes/index.js`. Another **78 alumni and mentorship routes** live in `server/src/routes/alumniRoutes.js`. Grouped by module, the features are:

Area	What users can do	Main backend controller
Accounts and security	Sign in, per-device sessions, change password, login history, admin security logs	`server/src/controllers/authController.js`, `securityController.js`
Community	Announcements, clubs, events (with registration and a waitlist), discussions, reports and moderation, global search	`announcementController.js`, `clubController.js`, `eventController.js`, `discussionController.js`, `reportController.js`, `searchController.js`
Academics	Timetables, subjects, exams, holidays, study materials	`timetableController.js`, `examController.js`, `holidayController.js`, `studyMaterialController.js`
Attendance	Student attendance, faculty attendance, OD and leave requests, corrections, reports, low-attendance monitor	`attendanceController.js`, `staffAttendanceController.js`, `odLeaveController.js`, `attendanceMonitorController.js`

Area	What users can do	Main backend controller
Campus life	Gate passes (with parent OTP and QR), lost and found, complaints with escalation, SOS alerts, hostel summary	`gatePassController.js`, `lostFoundController.js`, `complaintController.js`, `sosController.js`, `hostelController.js`
Communication	Private and group chat, notifications, push	`chatController.js`, `notificationController.js`, `server/src/utils/notify.js`
Transport	Live bus map, routes, stops, trips, ETA, GPS tracker webhook	`transportController.js`, `server/src/services/transport/`
AI	"JNN Study Assistant" over uploaded study material (Groq)	`aiController.js`, `server/src/services/ai/`
Analytics and admin	Dashboards, user management, role assignments, achievements, alumni network	`analyticsController.js`, `adminController.js`, `adminRoleAssignmentController.js`, `achievementController.js`, `server/src/controllers/alumni/`

Guide 05 walks through each module in detail.

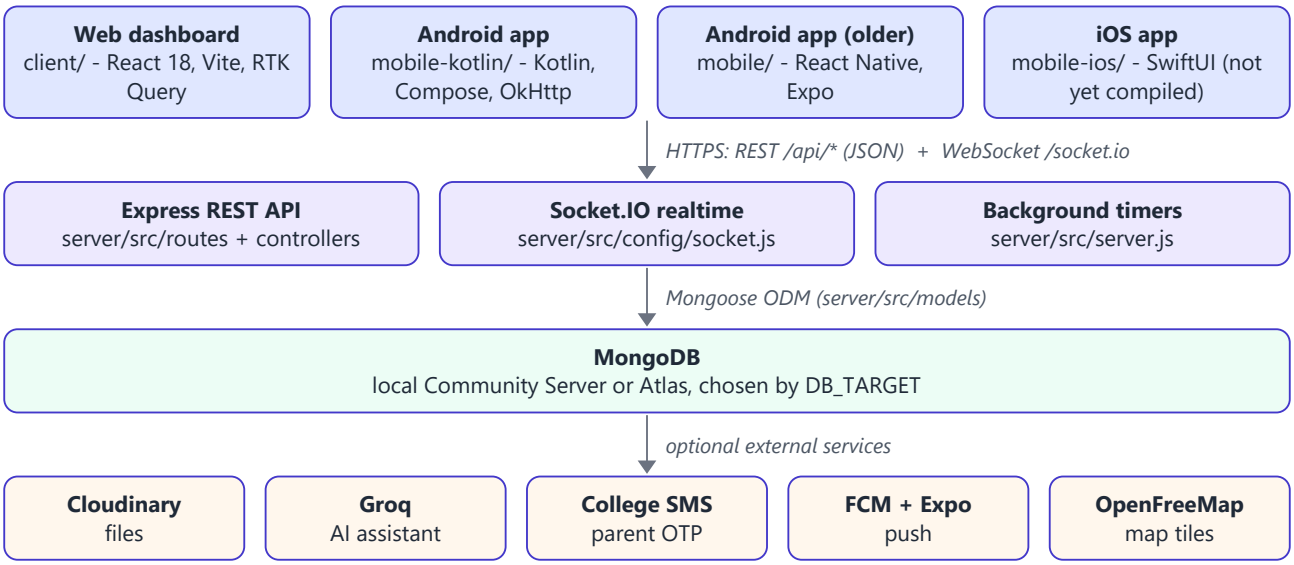
The repository at a glance

```
CMP/                          (repository root)
|-- package.json              root scripts: install:all, dev, seed, build, start, test
|-- render.yaml               Render.com deployment blueprint (one web service)
|-- .github/workflows/        CI, secret scan, keep-alive ping, OWASP ZAP baseline
|-- scripts/verify-bundle.mjs  scans the built web bundle for leaked secrets
|-- server/                   Node.js + Express + Socket.IO + Mongoose backend
|   |-- src/server.js         process entry: connect DB, start HTTP + sockets + timers
|   |-- src/app.js            Express app: security middleware, /api, static web app
|   |-- src/config/           env.js, db.js, socket.js
|   |-- src/routes/           index.js (all routes + validation), alumniRoutes.js
|   |-- src/middleware/       auth, CSRF/origin guard, rate limits, validation, errors
|   |-- src/controllers/      request handlers per module
|   |-- src/services/         larger business logic (attendance, transport, AI, OTP, PDF)
|   |-- src/models/           59 Mongoose models (one file per collection)
|   |-- src/utils/            helpers: http, tokens, notify, storage, permissions ...
|   |-- src/seed/             demo data loaders (seed.js WIPES the dev database)
|   |-- scripts/              one-off maintenance and smoke-test scripts
|   `-- tests/                80 integration test files (node:test)
|-- client/                   React 18 + Vite web dashboard
|   |-- src/main.jsx          React entry: Redux Provider + BrowserRouter
|   |-- src/App.jsx           route table and route guards
|   |-- src/services/api.js   RTK Query: every API endpoint the web app calls
|   |-- src/pages/            one folder per module
|   `-- tests/                5 unit test files (node:test)
|-- mobile/                   older Android client: React Native + Expo SDK 57
|-- mobile-kotlin/            native Android client: Kotlin + Jetpack Compose
|-- mobile-ios/               native iOS client: Swift + SwiftUI (first phase, not compiled)
`-- docs/                     screenshots, VPS setup guide, this handbook
```

> **Warning**
> `npm run seed` (which runs `server/src/seed/seed.js`) deletes the `campusconnect` database and loads demo data. Run it only on a local development database, never on one with real data.

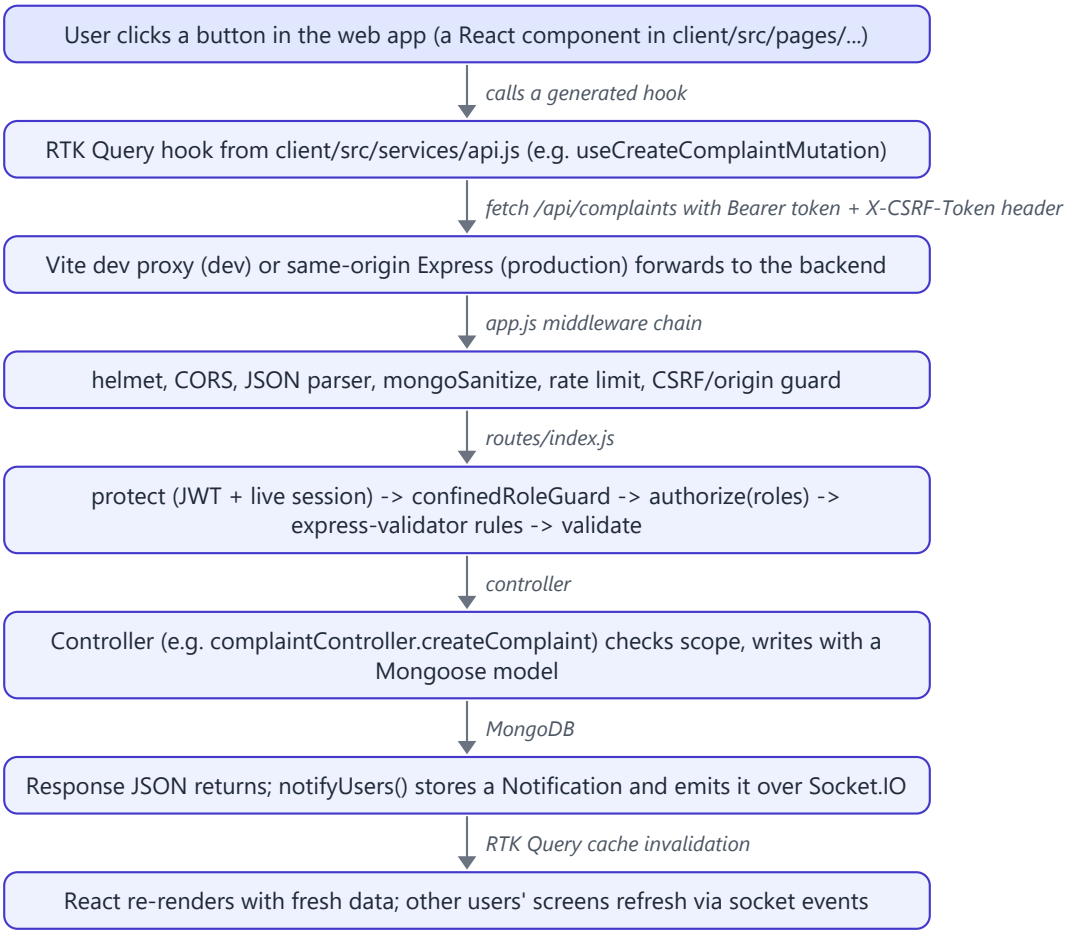
Architecture

The big picture



The key idea: **the server is the only authority**. Every client is "just a screen". All rules about roles, departments, ownership and validation run on the server. When the web app hides a button from a student, that is only cosmetic. The server refuses the request anyway.

How a request moves through the system



Guide 02 and Guide 03 trace this exact path in code, file by file.

How the frontend talks to the backend

One base URL. The web client always calls relative URLs starting with `/api`. In development, `client/vite.config.js` proxies `/api`, `/uploads` and `/socket.io` to `http://localhost:5000` (or `VITE_API_PROXY`). In production, Express serves the built React app from `client/dist` *and* the API from the same origin. The browser therefore never needs CORS for the web app.

One API file. `client/src/services/api.js` defines the whole web API with Redux Toolkit Query (`createApi`), about 345 endpoints. Each endpoint becomes a hook, for example `useGetDashboardQuery` or `useCreateComplaintMutation`. Components never call `fetch` directly for normal data.

Every request carries credentials (see `prepareHeaders` in `api.js`):

- `Authorization: Bearer <accessToken>`. The short-lived access token lives only in Redux memory (`client/src/features/authSlice.js`), never in localStorage.
- `X-CSRF-Token`. This copies the readable CSRF cookie (double-submit protection, see `server/src/middleware/security.js`).
- `credentials: 'include'`. This makes the browser send the httpOnly refresh cookie, which is scoped to `/api/auth`.

Automatic re-login. If any call returns `401`, `baseQueryWithReauth` calls `/api/auth/refresh` once (shared by all waiting requests), stores the new access token and replays the request. If the refresh fails, the user is logged out. A `401` with `reason: 'idle'` is final: the web idle rule ended the session.

Real-time. `client/src/services/socket.js` opens one Socket.IO connection with the access token. `client/src/components/layout/AppLayout.jsx` maps server events to RTK Query cache tags (`LIVE_TAGS`). When `announcement:changed` arrives, for example, the `Announcement` and `Dashboard` caches are refreshed over REST. Data is always re-read through REST, so each user's permissions are applied again.

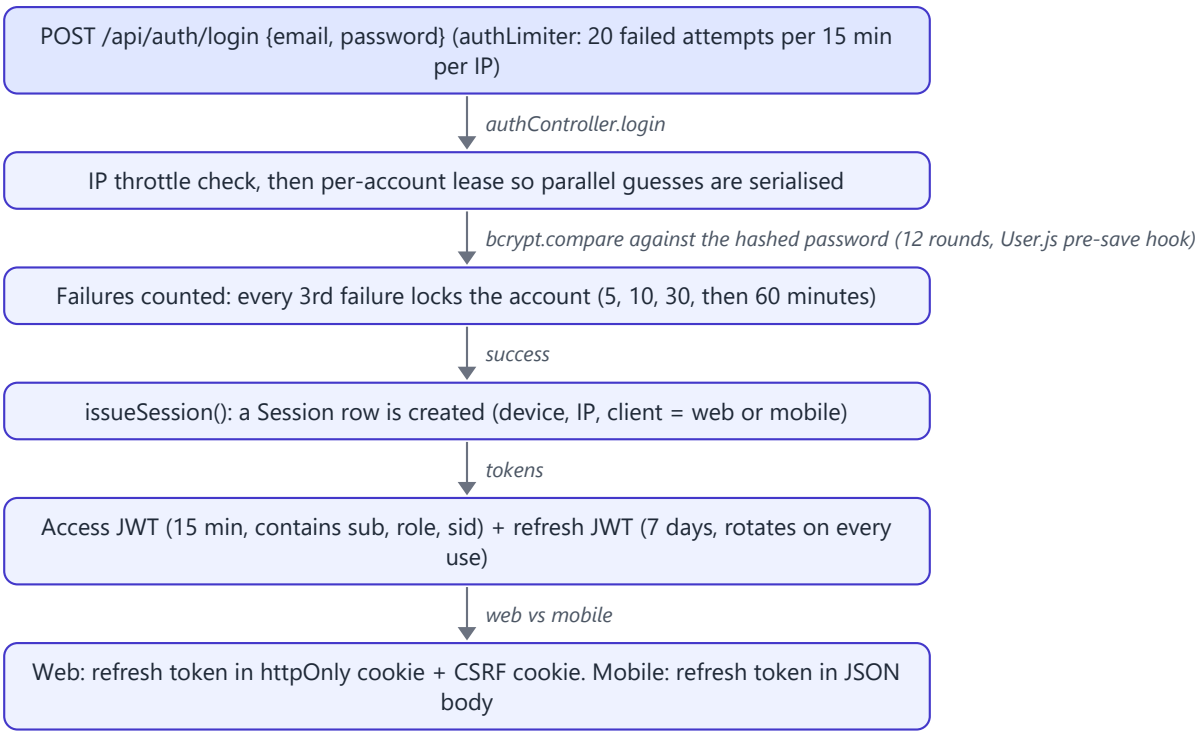
How the backend talks to the database

- `server/src/config/env.js` decides which MongoDB to use. `DB_TARGET=local` (the default) uses `MONGO_URI`, which defaults to a local `campusconnect` database. `DB_TARGET=atlas` uses `MONGO_URI_ATLAS`.
- `server/src/config/db.js` exports `connectDB()`. It retries 5 times, 3 seconds apart, then calls `syncIndexes()` on every model so the declared indexes exist. If every attempt fails, the process exits with a helpful message.
- Each collection has one model file in `server/src/models/`. `User.js` is the central model. Almost every other model references it with `ref: 'User'`.
- Controllers use Mongoose directly (`User.find`, `Complaint.create`, `aggregate` pipelines). There is no separate repository layer.

> **Evidence status**
> **PostgreSQL is not used anywhere in the code.** No PostgreSQL driver, ORM or SQL schema exists in any `package.json` or source file. Guide 04 treats a PostgreSQL migration strictly as **planned future work**.

Authentication and authorization

Signing in (authentication)



Important details, all in `server/src/utils/tokens.js` and `server/src/middleware/auth.js`:

- An access token is accepted only if its signature is valid (HS256 only), its `sid` names a live `Session` row, and the user is still active. Revoking a session (logout, "sign out other devices", password reset) therefore cuts off its access tokens at once.
- Refresh tokens **rotate**. Replaying an old refresh token is treated as theft and revokes the whole session.
- A client counts as "mobile" only if it sends `x-client-platform: mobile` *and* no `Sec-Fetch-Site` header. Browsers always send that header and page scripts cannot remove it, so a browser cannot pretend to be the app.
- **Web idle rule.** For web sessions of roles that are not exempt, 2 minutes without reported activity ends the session (`server/src/utils/sessionIdle.js`). The exempt roles are faculty, hod, admin, principal, ao, dean, chairman, warden and security. So students, club admins, COE, office, transport and alumni web users get the 2-minute rule. Mobile sessions use a 7-day idle rule.

> **Common mistake**
> The README says the account locks after "5 failures for 15 minutes". The current code in `authController.login` locks after every **3** failures, for 5, 10, 30 and then 60 minutes. When documentation and code disagree, **the code wins**. Fix the documentation.

Permissions (authorization)

There are **15 roles**, defined in `ROLES` in `server/src/constants.js`: student, club_admin, faculty, hod, principal, admin, security, dean, ao, chairman, warden, alumni, coe, office and transport.

Authorization happens in four layers. Learn them, because a bug can sit in any one of them:

Layer	Where	What it decides
1. `protect`	`server/src/middleware/auth.js`	Are you signed in with a live session? (401 if not)
2. `confinedRoleGuard`	`server/src/middleware/coeGuard.js`	Deny-by-default fences for `coe`, `office` and `transport`: they may only call their own module's paths
3. `authorize(...roles)`	per route in `server/src/routes/index.js`	Is your role allowed on this route at all? (403)
4. Scope checks in controllers	e.g. `timetableController.js`, `utils/academicScope.js`, `utils/peopleScope.js`	Is this record in *your* department, class or ownership?

Role lists such as `TIMETABLE_WRITE_ROLES` or `TRANSPORT_VIEW_ROLES` live in `server/src/constants.js`. The web app keeps matching lists in `client/src/utils/constants.js`, but those only control what is shown.

Real-time communication (Socket.IO)

`server/src/config/socket.js` (`initSocket`) attaches Socket.IO to the same HTTP server. Every connection must present an access token. The same `authenticateAccessToken()` used by REST checks it.

On connect, each socket joins three rooms: `user:<id>`, `session:<sid>` and `role:<role>`. Further rooms are joined only after a database check:

Client event	Room joined	Check performed
`chat:join`	`chat:<conversationId>`	Membership in the `Conversation`, plus chat scope for private chats
`discussion:join`	`discussion:<id>`	Hidden threads only for moderators

Client event	Room joined	Check performed
`transport:subscribe`	`bus:<id>` or `transport:all`	`viewerScope` / `canViewBus` in `services/transport/access.js`

The server pushes events such as `notification`, `presence:update`, `chat:typing`, `bus:location` and many `*:changed` / `*:updated` signals. The design rule: **REST does every read and write. Sockets only say "something changed"** (plus chat traffic and live bus positions).

All notifications go through one function, `notifyUsers()` in `server/src/utils/notify.js`. It drops users who switched a category off, stores the `Notification` (de-duplicated with `eventKey`), emits it live to `user:<id>`, and then attempts push delivery (`server/src/services/pushService.js`).

Mobile and iOS architecture

How the apps share the backend

All mobile clients call **the same `/api` routes and Socket.IO events** as the web app. The only differences are on the session side:

- They send `x-client-platform: mobile`, so the refresh token comes back in the JSON body instead of a cookie.
- They store the refresh token in encrypted device storage: Android Keystore AES-GCM in `mobile-kotlin/app/src/main/java/com/vexon/campus/core/SecureStore.kt`, and the iOS Keychain in `mobile-ios/VEXON/Core/Authentication/KeychainStore.swift`.
- They are exempt from the CSRF check, because native apps carry no browser cookies, and from the web 2-minute idle rule.

The three mobile code bases

Folder	Technology	Status (evidence)
`mobile/`	React Native, Expo SDK 57, expo-router, RTK Query	**Implemented, older client.** README section 8 documents it. Last commit touching it: 12 Oct 2026. The Kotlin README calls the Kotlin app "a rewrite of the Expo app", and the Expo app "still builds". Its 13 unit tests (`mobile/tests/`) passed during this audit; no build was attempted
`mobile-kotlin/`	Kotlin 2.2, Jetpack Compose, OkHttp, socket.io-client-java, MapLibre, CameraX + ML Kit, Play Services location, FCM	**Implemented and actively developed.** It has 8 unit-test files (50 tests), which passed during this audit (see Guide 07). Building APKs and running on a device were not done in this audit
`mobile-ios/`	Swift 6, SwiftUI, a local Swift package `VEXONKit`, MapLibre iOS, Socket.IO Swift	**Unable to verify / partially implemented.** Its own README says: "Nothing here has been compiled yet … Not production-ready". The folder has **no commits** in git (it is untracked), and it needs a Mac with Xcode 16+

The Kotlin app maps one-to-one onto the Expo structure (see the table in `mobile-kotlin/README.md`): `core/Http.kt` plays the role of RTK Query's base query, `core/Realtime.kt` plays the role of `socket.js`, and `nav/Nav.kt` uses the same paths as the web routes, so server links like `/gate-pass/<id>` work everywhere.

External services

Service	Used for	Config (names only)	Status
MongoDB Atlas / local MongoDB	All data	`DB_TARGET`, `MONGO_URI`, `MONGO_URI_ATLAS`	Implemented
Cloudinary	Uploaded images and files survive restarts	`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	Optional. Without it, production stores files in MongoDB (`StoredFile`) and development writes them to `server/uploads`
Groq API	JNN Study Assistant (LLM, optional web search)	`GROQ_API_KEY`, `GROQ_MODEL`, `AI_*`	Optional. The assistant is disabled without a key
College SMS gateway	Parent OTP and exit SMS for gate passes	`OTP_SMS_ENABLED`, `OTP_PROVIDER`, `SMS_*`	Off by default in `.env.example`. `render.yaml` turns it on for the deployed service
Firebase Cloud Messaging	Push to the Kotlin app	`FCM_SERVICE_ACCOUNT` / `FCM_SERVICE_ACCOUNT_FILE`	Optional
Expo push	Push to the legacy Expo app	`EXPO_ACCESS_TOKEN` (optional)	Legacy path in `pushService.js`
OpenFreeMap (default)	Map style and tiles for the bus map	`MAP_STYLE_URL`, `MAP_CSP_ORIGINS`	Implemented. Any provider can be configured
SMTP (nodemailer)	Security alert emails (e.g. lockout)	`SMTP_*`	Optional. Email password reset is retired (returns 410)

Important dependencies

Package	Side	Why VEXON uses it
express 4	server	HTTP routing and middleware
mongoose 8	server	MongoDB schemas, validation, queries
socket.io 4	server + clients	Real-time events
jsonwebtoken, bcryptjs	server	Access/refresh tokens, password hashing
helmet, cors, express-rate-limit, express-mongo-sanitize	server	HTTP security headers, origin allow-list, throttling, NoSQL-injection defence
express-validator	server	Request validation rules in `routes/index.js`
multer, file-type	server	Uploads held in memory and checked by magic bytes
groq-sdk, unpdf	server	AI chat; extracting text from PDFs for the assistant
qrcode, nodemailer, morgan, compression, cookie-parser, dotenv	server	Gate-pass QR, email, dev request logging, gzip, cookies, `.env` loading
react 18, react-router-dom 6	client	UI and routing
@reduxjs/toolkit, react-redux	client	Store + RTK Query data fetching
react-hook-form	client	Forms and client-side validation
tailwindcss, lucide-react, recharts, react-hot-toast	client	Styling, icons, charts, toasts
maplibre-gl 6.11.2, jsqr	client	Bus map, QR scanning in the browser

Running VEXON locally

Prerequisites from `README.md`: Node.js 18.18+ (this machine has Node 24) and MongoDB Community Server 6.0+ running locally.

```bash
npm run install:all                # root + server + client dependencies
copy server\.env.example server\.env   # Windows; then edit secrets (never commit .env)
npm run seed                       # DEV ONLY: wipes and reloads the campusconnect database
npm run dev                        # API on :5000 and web on :5173 together (concurrently)
```

Open `http://localhost:5173`. The demo accounts are listed in `README.md` (seed data only). To start pieces separately: `npm run dev --prefix server` (nodemon) and `npm run dev --prefix client` (Vite).

For production on one port, run `npm run build` and then `npm start`. Express serves `client/dist` plus the API on port 5000.

What the server does when it starts

`server/src/server.js` (`start()`) runs these steps in order:

1. `connectDB()`. It connects to MongoDB with retries and syncs indexes.
2. `syncSubjects()`. It adds any missing subjects from `server/src/data/timetableData.js`.
3. `normaliseBreakPeriods()`. It renumbers old timetable days where a break took a period number.
4. `createApp()`, `http.createServer` and `initSocket()`.
5. Timers: the gate-pass expiry sweep (every 5 minutes), `startGatePassCodeQueue`, `startKeepAlive` (Render free plan), `startIdleSessionSweeper` (30 s), `startAlumniSchedulers`, `warmAttendanceTotals`, `startReminders` (every minute) and `startTripSweeper` (bus trips, every minute).
6. `server.listen(PORT)`, plus graceful shutdown on SIGINT and SIGTERM.

Known documentation and code mismatches

Found during the audit. Each one is worth correcting in `README.md` when you have time.

Topic	README says	Code says (wins)
Account lockout	5 failures, 15 minutes	Every 3 failures; 5, 10, 30, 60 minutes (`authController.js`)
Idle exemptions	Chairman, security, admin, principal	Also faculty, hod, ao, dean, warden (`utils/sessionIdle.js`)
Render service	`vexon-api`, `https://vexon-api.onrender.com`	`jnn-api`, `https://jnn.onrender.com` (`render.yaml`)
"Android app"	The Expo app in `mobile/`	Two Android apps exist; the Kotlin one is the actively developed one
Production uploads	Local disk, or Cloudinary	Without Cloudinary, production stores files in MongoDB (`utils/storage.js`, `StoredFile`)
Test count	"637 at last run"	Re-counted in this audit; see Guide 07

Glossary

Term	Plain-English meaning
API	A set of URLs a program calls to read or change data
REST	A style of API: resources at URLs, verbs GET/POST/PUT/PATCH/DELETE
Endpoint / route	One method + URL pair, e.g. `POST /api/complaints`
Middleware	A function that runs before (or after) a route handler, e.g. `protect`
Controller	The function that does a route's real work
Model / schema	Mongoose definition of a collection's fields and rules
ODM	Object-Document Mapper: Mongoose maps JS objects to MongoDB documents
JWT	JSON Web Token: a signed string proving who you are
Access / refresh token	Short-lived proof sent on every request / longer-lived ticket to get a new access token
Session	A `Session` row per signed-in device; revoking it signs that device out
CSRF	Cross-Site Request Forgery: another site tricking your browser into a write; blocked by the double-submit token
CORS	Browser rule about which origins may call an API
RBAC	Role-Based Access Control
Socket.IO room	A named group of connections that receive the same events
RTK Query	Redux Toolkit's data-fetching and caching layer
Cache tag	A label (e.g. `'Complaint'`) used to refresh cached data after a change
Hook	A React function starting with `use` (e.g. `useState`, `useGetDashboardQuery`)
Seed	A script that fills a database with demo data
Environment variable	Configuration read from `.env` / the host, never committed
CI	Continuous Integration: GitHub Actions running tests on every push or PR

Check your understanding

Answer these without AI. Each answer is somewhere in this guide or in the files it names.

1. Which file decides which MongoDB database the server connects to, and which variable switches it to Atlas?
2. A student opens the browser console and calls `POST /api/timetable`. Which middleware or check refuses them, and with which status code?
3. Why can a stolen *old* refresh token not be used twice?
4. What are the four layers of authorization? Give one file for each.
5. How does the web app learn that an announcement changed, and why does it then re-fetch over REST instead of using the socket payload?
6. Which header makes the backend treat a client as the mobile app, and why can a browser not fake it?
7. In production without Cloudinary, where does an uploaded image go?
8. Which three mobile folders exist, and what is the status of each?
9. Name four timers `server.js` starts.
10. What happens to data if you run `npm run seed`?

Source references

README.md · package.json · render.yaml · server/package.json · server/src/server.js · server/src/app.js · server/src/config/env.js · server/src/config/db.js · server/src/config/socket.js · server/src/routes/index.js · server/src/routes/alumniRoutes.js · server/src/middleware/auth.js · server/src/middleware/coeGuard.js · server/src/middleware/security.js · server/src/utils/tokens.js · server/src/utils/sessionIdle.js · server/src/utils/notify.js · server/src/utils/storage.js · server/src/constants.js · server/src/models/User.js · client/package.json · client/vite.config.js · client/src/main.jsx · client/src/App.jsx · client/src/services/api.js · client/src/services/socket.js · client/src/components/layout/AppLayout.jsx · mobile-kotlin/README.md · mobile-ios/README.md · mobile/package.json